

Republic of Cameroon

Peace-Work-Fatherland

Ministry of Higher Education

University of Buea

**Faculty of Engineering and
Technology**

**Department of Computer
Engineering**



République du Cameroun

Paix-Travail-Patrie

**Ministère de l'enseignement
supérieures**

Université de Buea

**Faculty of Engineering and
Technology**

**Department of Computer
Engineering**

CAR FAULT DIAGNOSTIC APP: DATABASE DESIGN AND IMPLEMENTATION

Course Instructor : Dr Nkemeni Valery

Course code : CEF440

Group Members

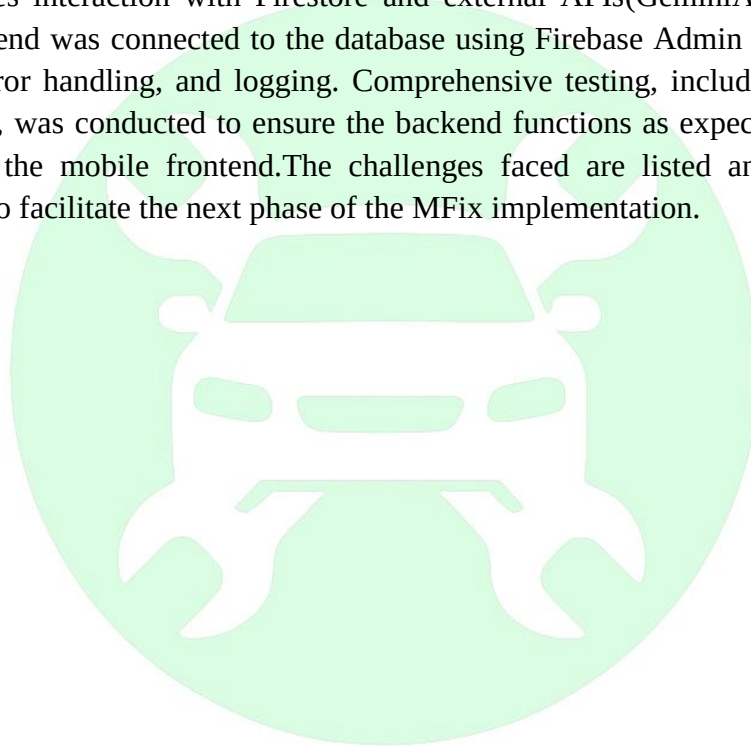
- | | |
|---|-----------------|
| 1- Angi Atangwa Valerine | FE22A148 |
| 2- Av'Nyhbeuhkonh Ezekiel Hosana | FE22A160 |
| 3- Beng Dwe Meinoff | FE22A173 |
| 4- Jong Turkson Junior | FE22A228 |
| 5- Nyuyesemo Calglain | FE22A287 |

Table of Contents

ABSTRACT	3
1. INTRODUCTION	4
1.1 PURPOSE OF DATABASE DESIGN.....	4
1.2 TECHNOLOGIES USED	4
2. DATA ELEMENTS.....	5
2.1 IDENTIFICATION OF KEY ENTITES.....	5
2.2 ATTRIBUTES OF EACH ENTITY	6
2.3 SAMPLE DATA ELEMENTS	6
3. CONCEPTUAL DIAGRAM	9
3.1 NORMALIZATION OBSERVATIONS	10
4. ENTITY-RELATIONSHIP (ER) DIAGRAM	10
4.1 ER DIAGRAM OVERVIEW.....	10
4.2 DESCRIPTION OF ENTITIES AND RELATIONSHIPS	10
5- Database Implementation.....	11
5.1 – Database Management System.....	12
5.2 – Schema Definition.....	12
5.4 Table Structures and Constraints.....	14
5.4 Indexing and Optimization Considerations.....	15
6.1 Backend Framework and Tools.....	16
6.2 API Design and Routes	16
7- CONNECTING DATABASE TO BACKEND.....	20
7.1 Database Configuration.....	20
7.2 Connection Logic in Backend	20
7.3 Error Handling and Logging	20
8. TESTING AND VALIDATION.....	21
9- CHALLENGE FACED AND SOLUTIONS.....	23
10- FURTHER ENHANCEMENT AND PENDING TASKS	24
11- CONCLUSION.....	24
REFERENCES.....	25

ABSTRACT

This report presents the complete backend and database design of the MFix Car Fault Diagnostic Application, a mobile system developed to help users identify vehicle issues through image and sound diagnostics. It begins with an introduction to the app's purpose and the technologies used, followed by the identification of key data entities and their attributes. A conceptual design was developed to illustrate relationships between entities, supported by an ER diagram and normalization principles to ensure data integrity. For implementation, Firebase Firestore was chosen as the DBMS due to its scalability and real-time capabilities, with a defined schema and indexing strategies to optimize performance. The backend was built using Node.js and Firebase, with RESTful API routes created for user registration, diagnostics, requests, and results. CRUD operations were implemented for all major collections, and sample code demonstrates interaction with Firestore and external APIs(GeminiApi, Youtube API, etc....) The backend was connected to the database using Firebase Admin SDK, with proper configuration, error handling, and logging. Comprehensive testing, including API calls and integration flows, was conducted to ensure the backend functions as expected and integrates seamlessly with the mobile frontend.The challenges faced are listed and the phase was conducted so as to facilitate the next phase of the MFix implementation.



1. INTRODUCTION

This document presents the database design and implementation for Mfix, a mobile application developed to assist users in diagnosing car faults efficiently.

1.1 PURPOSE OF DATABASE DESIGN

The purpose of the database design for Mfix is to create a structured and efficient data management system that supports the core functionalities of the car fault diagnostic mobile application. The database is intended to:

- Store and organize vehicle-related data, user data, and all other entity-related data.
- Maintain a comprehensive repository of car fault codes, symptoms, and potential causes based on standardized diagnostic information.
- Track user interactions and diagnostic history, enabling personalized recommendations and future reference.
- Support fast and accurate retrieval of diagnostic results, ensuring users receive timely feedback on potential issues.
- Enable seamless integration with the backend, allowing the mobile app to interact with the stored data in real time for querying, updating, and reporting.

This database design serves as the foundation for delivering a reliable, scalable, and user-friendly diagnostic tool that empowers users to better understand and manage their vehicle health.

1.2 TECHNOLOGIES USED

Technology	Purpose and Role in MFix Application
Firestore	A cloud-hosted NoSQL database used to store all app data including users, vehicles, diagnostics, and requests in a flexible and scalable structure.
Authentication	Manages user sign-up, login, and role-based access control with secure token handling.
Admin SDK (Node.js)	Enables secure server-side access to Firestore, Authentication, and Storage from the custom backend built using Node.js.
React Native (Expo)	A cross-platform framework used to build the mobile frontend of MFix, allowing fast development for both Android and iOS.
Gemini API	Used for AI-based interpretation of dashboard images and engine sound diagnostics through generative AI responses.

Technology	Purpose and Role in MFix Application
YouTube Data API v3	Integrated to retrieve educational and repair-related videos based on diagnostic results to guide users visually.
Postman	Utilized to test and validate backend API endpoints by simulating requests and reviewing responses during development.

2. DATA ELEMENTS

Data elements are the smallest units of information stored in a database. They represent individual pieces of data that describe an attribute of an entity.

2.1 IDENTIFICATION OF KEY ENTITIES

Some key entities for our car fault diagnostic app are :

Key Entity	Description
User	Represents individuals who register with the system. Users can own vehicles, receive notifications, request mechanics, and access repair tutorials.
Vehicle	Owned by a user, it represents any registered car in the system. Vehicles are associated with diagnostics, maintenance, and repair history.
Diagnostic	A diagnostic event performed on a vehicle. Each diagnostic may include findings from dashboard systems or sound-based analysis.
DashboardDiagnosis	Details diagnostic results based on the vehicle's dashboard error codes.
SoundDiagnosis	Captures information about diagnostic analysis from sound patterns.
Maintenance	Logs maintenance activities performed on a vehicle.
Mechanic	Registered professionals who perform diagnostics or maintenance and may receive service requests from users.
RepairTutorial	Instructional content linked to diagnostics, intended to help users resolve issues themselves.
Notification	System-generated messages sent to users regarding diagnostics or requests.

MechanicRequest	Captures a user's request for mechanic assistance, linking user, vehicle, and mechanic.
-----------------	---

2.2 ATTRIBUTES OF EACH ENTITY

An **attribute** is a **property or characteristic** of an **entity** in a database.

- It provides **specific details** about the entity.
- In database tables, attributes are represented as **columns**.
- Each attribute holds a **single piece of information** about the entity.

Below are the attributes of each entity we have ;

Entity	Attributes
User	UserID (PK), Name, Email, PhoneNumber, RegistrationDate, Role
Vehicle	VehicleID (PK), UserID (FK), Make, Model, Year, VIN, RegistrationDate
Diagnostic	DiagnosticID (PK), VehicleID (FK), Date, Type, Status
DashboardDiagnosis	DashboardDiagnosisID (PK), DiagnosticID (FK), ErrorCode, Description, Severity
SoundDiagnosis	SoundDiagnosisID (PK), DiagnosticID (FK), SoundPattern, Frequency, Severity
Maintenance	MaintenanceID (PK), VehicleID (FK), Date, Type, Technician
Mechanic	MechanicID (PK), Name, Speciality, ContactInfo, Availability
RepairTutorial	TutorialID (PK), DiagnosticID (FK), Title, Content, VideoLink, DifficultyLevel
Notification	NotificationID (PK), UserID (FK), VehicleID (FK), Message, Date, Type
MechanicRequest	RequestID (PK), UserID (FK), MechanicID (FK), VehicleID (FK), RequestDate, Status

2.3 SAMPLE DATA ELEMENTS

Entity: User

Attribute	Sample Value
UserID	1
Name	Alice Johnson

Email	alice@example.com
PhoneNumber	+23764567890
RegistrationDate	2025-01-10
Role	Owner

Entity: Vehicle

Attribute	Sample Value
VehicleID	101
UserID	1
Make	Toyota
Model	Corolla
Year	2020
VIN	1HGCM82633A123456
RegistrationDate	2025-01-11

Entity: Diagnostic

Attribute	Sample Value
DiagnosticID	201
VehicleID	101
Date	2025-06-01
Type	Routine Check
Status	Completed

Entity: DashboardDiagnosis

Attribute	Sample Value
DashboardDiagnosisID	301
DiagnosticID	201
ErrorCode	P0420
Description	Catalyst Efficiency Below Threshold
Severity	High

Entity: SoundDiagnosis

Attribute	Sample Value
SoundDiagnosisID	401
DiagnosticID	201
SoundPattern	Knocking
Frequency	High
Severity	Moderate

Entity: Maintenance

Attribute	Sample Value
MaintenanceID	501
VehicleID	101
Date	2025-05-15
Type	Oil Change
Technician	Tom Nguyen

Entity: Mechanic

Attribute	Sample Value
MechanicID	601
Name	Jane Smith
Speciality	Electrical Systems
ContactInfo	jane@fixcars.com
Availability	Weekdays 9–5

Entity: RepairTutorial

Attribute	Sample Value
TutorialID	701
DiagnosticID	201
Title	Fixing a P0420 Error
Content	Step-by-step guide...
VideoLink	https://youtu.be/example
DifficultyLevel	Intermediate

Entity: Notification

Attribute	Sample Value
NotificationID	801
UserID	1
VehicleID	101
Message	Upcoming maintenance
Date	2025-06-05
Type	Reminder

Entity: MechanicRequest

Attribute	Sample Value
RequestID	901
UserID	1

MechanicID	601
VehicleID	101
RequestDate	2025-06-02
Status	Pending

3. CONCEPTUAL DIAGRAM

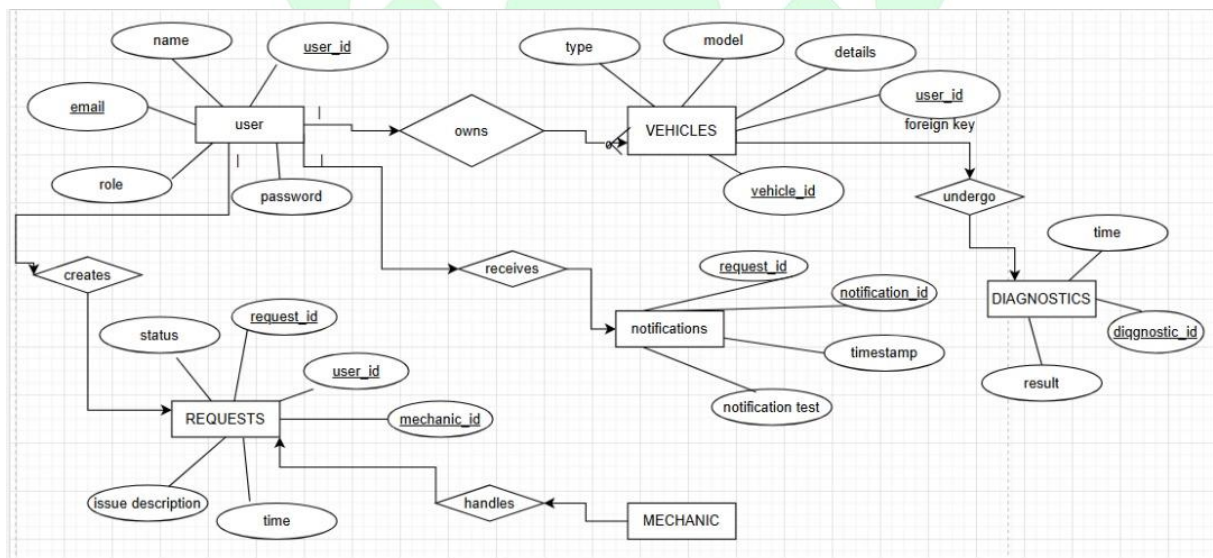
A conceptual diagram is a high-level, abstract representation of how data is structured and how different parts of a system relate to each other without worrying about implementation details like tables, data types, or programming languages.

Purpose of a Conceptual Diagram

- To visualize the core data entities and their relationships in a system.
- To provide a blueprint for database design before creating actual tables or writing backend code.
- To help developers, designers, and stakeholders understand the system's logic and data flow at a glance.

Below we have the conceptual diagram for our system

The conceptual diagram below shows the main entities, attributes and their various relationships



3.1 NORMALIZATION OBSERVATIONS

The design adheres to **3rd Normal Form (3NF)**:

- **1NF**: Each attribute has atomic values.
- **2NF**: All non-key attributes are fully dependent on their primary keys.
- **3NF**: No transitive dependencies exist among non-key attributes.

4. ENTITY-RELATIONSHIP (ER) DIAGRAM

4.1 ER DIAGRAM OVERVIEW

An **ER diagram** (Entity-Relationship diagram) is a visual representation used in database design to illustrate the structure of a database. It shows how different **entities** (things or objects) are related to each other.

Key components of an ER diagram:

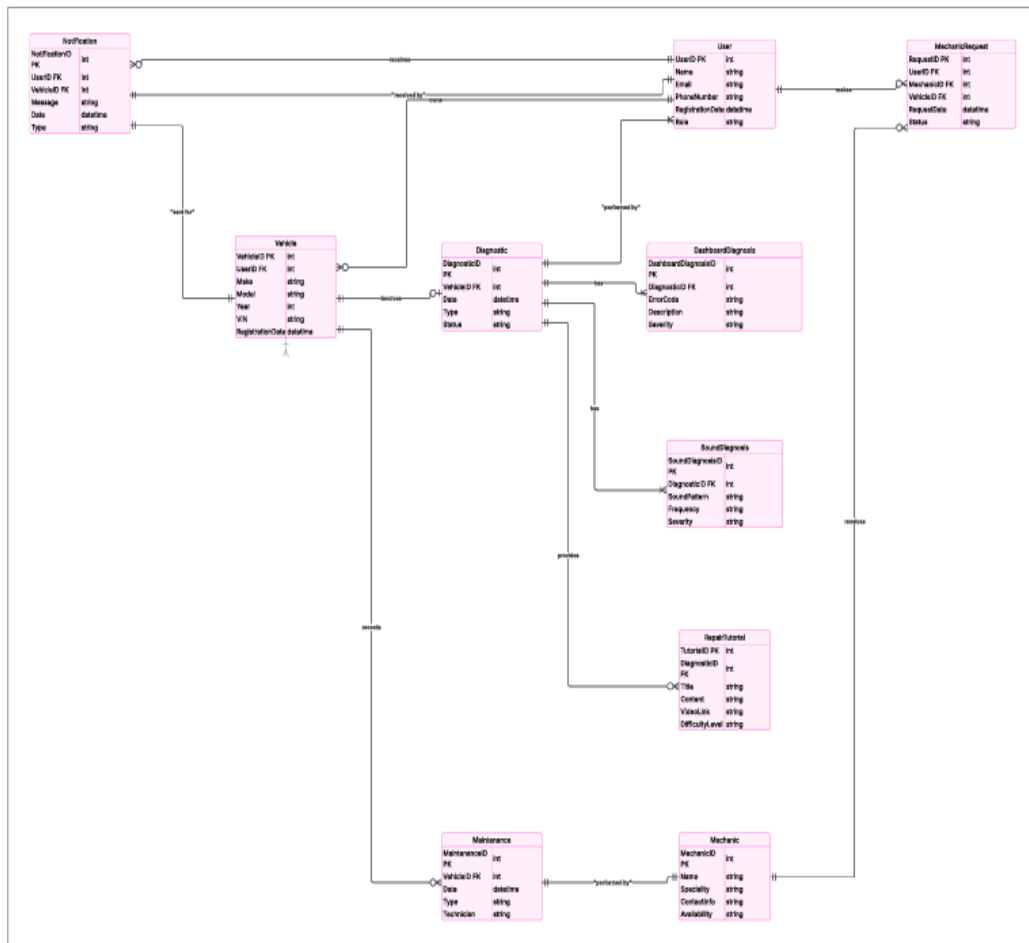
- **Entities**: Represent real-world objects or concepts.
- They are usually shown as rectangles.
- **Attributes**: Properties or details of an entity.
- Attributes are shown as ovals connected to their entity.
- **Relationships**: How entities are connected or related to one another. These are shown as diamonds between entities.
- **Cardinality**: Defines the number of instances of one entity related to instances of another (e.g., one-to-many, many-to-many).

ER diagrams are important because of many reasons such as ;

- To plan and design databases clearly before implementation.
- To help developers and stakeholders understand the data structure.
- To identify the relationships and constraints in the system.

4.2 DESCRIPTION OF ENTITIES AND RELATIONSHIPS

Below is the ER diagram for our system ;



5- Database Implementation

A **database** is an organized collection of structured information or data that is electronically stored and accessed. In software systems, databases are essential for storing user profiles, application data, logs, and more in a secure and scalable way. They allow our systems to retrieve, update, delete, and manage data efficiently through queries or APIs.

Database implementation here, refers to the practical process of creating and configuring the database system that supports application functionality. It includes selecting a database management system (DBMS), designing the structure (schema), and implementing rules, relationships, and data handling mechanisms.

For the MFix system, our goal was to implement a real-time, flexible, and scalable database that supports vehicle diagnostics, user roles, file uploads, and request tracking — all with minimum server maintenance.

5.1 – Database Management System

A **Database Management System (DBMS)** is software that allows users and applications to interact with a database. It provides tools to create, store, retrieve, secure, and manage data in a structured manner. A DBMS acts as a bridge between the application and the physical storage layer. A DBMS is important for it

- Ensures **data consistency** and **security**
- Manages **concurrent access** and real-time updates
- Supports **querying, indexing, and relationships**
- Reduces redundancy and improves **data organization**

For the backend of the MFix mobile diagnostic application, we selected **Firebase Cloud Firestore** as our Database Management System (DBMS). Firestore is a flexible, scalable NoSQL cloud database that enables real-time data synchronization across clients. The choice was motivated by the need for:

- Seamless integration with our React Native (Expo) frontend
- Real-time update capabilities
- Built-in security rules and authentication support
- Minimal server management via Firebase Admin SDK

To use firestore as the DBMS we went through the following process

- Each main entity (e.g., User, Vehicle, Request) became a **collection**
- Each instance of an entity is stored as a **document** with key-value fields
- We used **nested fields, document references, and timestamps** for relationships and filtering
- Firebase SDK was used on the frontend to interact with the Firestore database

5.2 – Schema Definition

A **schema** here refers to the structure of the database: the entities (tables/collections), the fields (columns/attributes), and the relationships between them. While relational databases use fixed schemas, NoSQL databases like Firestore use dynamic schemas — allowing documents to vary in structure — though consistency is still enforced logically.

Schema Used in MFix

Although Firestore is schema-less, we followed a **logical schema** to ensure consistency and reliable querying. Below is the schema:

Collection and structure is given below

users/

- └─ userId
 - └─ name, email, password, role, phone

vehicles/

- └─ vehicleId
 - └─ userId (ref), make, model, year

mechanics/

- └─ mechanicId
 - └─ userId (ref), location, availability, rating

diagnostics/

- └─ diagnosticId
 - └─ userId (ref), vehicleId (ref), type, status, resultId (ref), createdAt

results/

- └─ resultId
 - └─ faultName, explanation, recommendation, videoUrl, createdAt

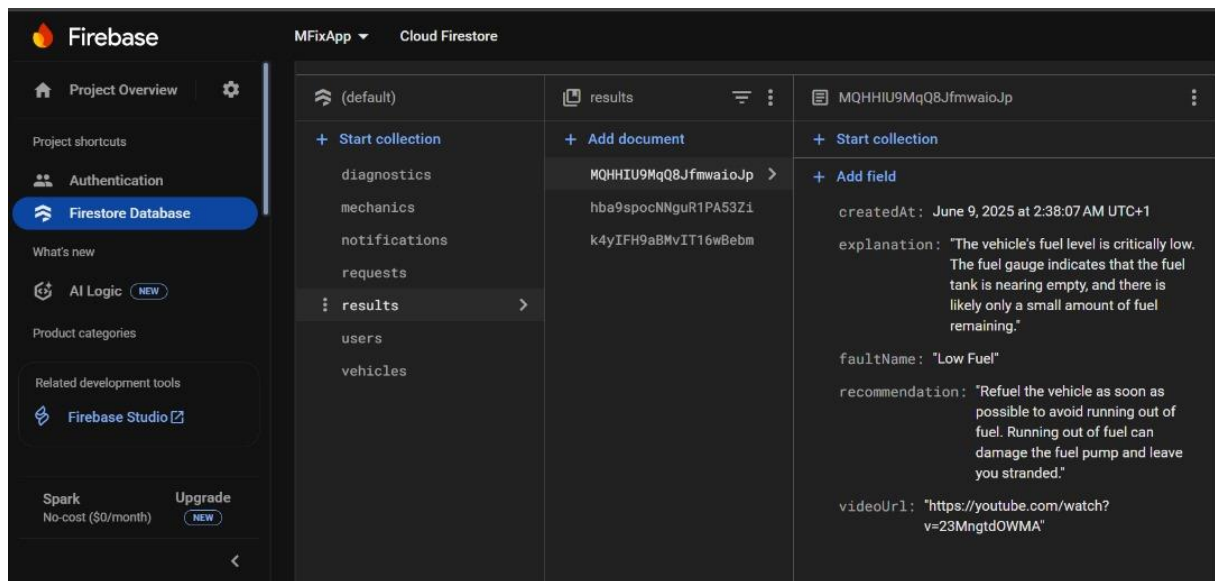
requests/

- └─ requestId
 - └─ userId (ref), mechanicId (ref), issue, status, createdAt

notifications/

- └─ notificationId
 - └─ userId (ref), message, resultId, read, timestamp

The picture below shows the firebase Console showing collections and sample documents.



Why This Schema?

- Enforces **logical relationships** between users, vehicles, and diagnostics
- Allows **efficient filtering and indexing**
- Reduces data redundancy (e.g., results are reused across users)
- Supports **role-based access** (user vs. mechanic)

5.4 Table Structures and Constraints

Although Firestore doesn't use SQL tables, we structured our collections with the following constraints:

Collection	Field	Constraint
users	role	must be either "user" or "mechanic"
vehicles	userId	must reference an existing user
diagnostics	vehicleId	must reference an existing vehicle
diagnostics	type	must be "image" or "sound"
requests	mechanicId	must reference a user with role "mechanic"

Collection	Field	Constraint
results	faultName	must match known diagnostic faults
notifications	read	boolean (true/false only)

These constraints were enforced using:

- Frontend validation
- Firebase Cloud Functions (for logic checks)
- Firestore Security Rules

5.4 Indexing and Optimization Considerations

Default Indexing

Firestore automatically indexes all single fields. However, for performance and querying, we created **composite indexes** for:

- diagnostics: `userId` + `createdAt`
- requests: `mechanicId` + `status`
- results: `faultName` + `createdAt`
- notifications: `userId` + `timestamp`

Optimization Practices Used

- **Denormalization:** We duplicated small data fields (e.g., mechanic name in requests) to reduce reads
- **Pagination:** Used `.limit()` and `.orderBy()` in queries to load data in chunks
- **Timestamps:** Stored `createdAt` and `updatedAt` for chronological ordering
- **Conditional fetching:** Diagnostics or results are only fetched when requested

This database design supports the dynamic and real-time needs of the MFix system. Using Firebase Firestore gave us a scalable, secure, and mobile-first backend without the overhead of managing a server. Our schema was carefully designed to reflect real-world relationships and ensure efficient data access across all parts of the application.

6- BACKEND IMPLEMENTATION

6.1 Backend Framework and Tools

The **backend** refers to the server-side of an application. It handles data processing, business logic, database interaction, and serves content to the frontend (client). It's the part users don't see but that powers everything behind the scenes.

Tools and Frameworks Used in MFix

- **Node.js:** A JavaScript runtime that lets us run backend code outside the browser.
- **Express.js:** A web framework for Node.js that helps create RESTful APIs.
- **Firebase Admin SDK:** Provides backend access to Firebase Authentication, Firestore, and Storage.
- **dotenv:** A Node.js module used to load environment variables securely from a `.env` file.
- **Postman:** Used to test API endpoints during development.

Together, these tools allowed us to implement a secure, serverless, and scalable backend tailored for a mobile app like MFix.

6.2 API Design and Routes

An **API (Application Programming Interface)** is a set of rules that allows different parts of an application to communicate. In our case, the frontend (React Native app) communicates with the backend using HTTP requests via API routes.

RESTful Routes Used

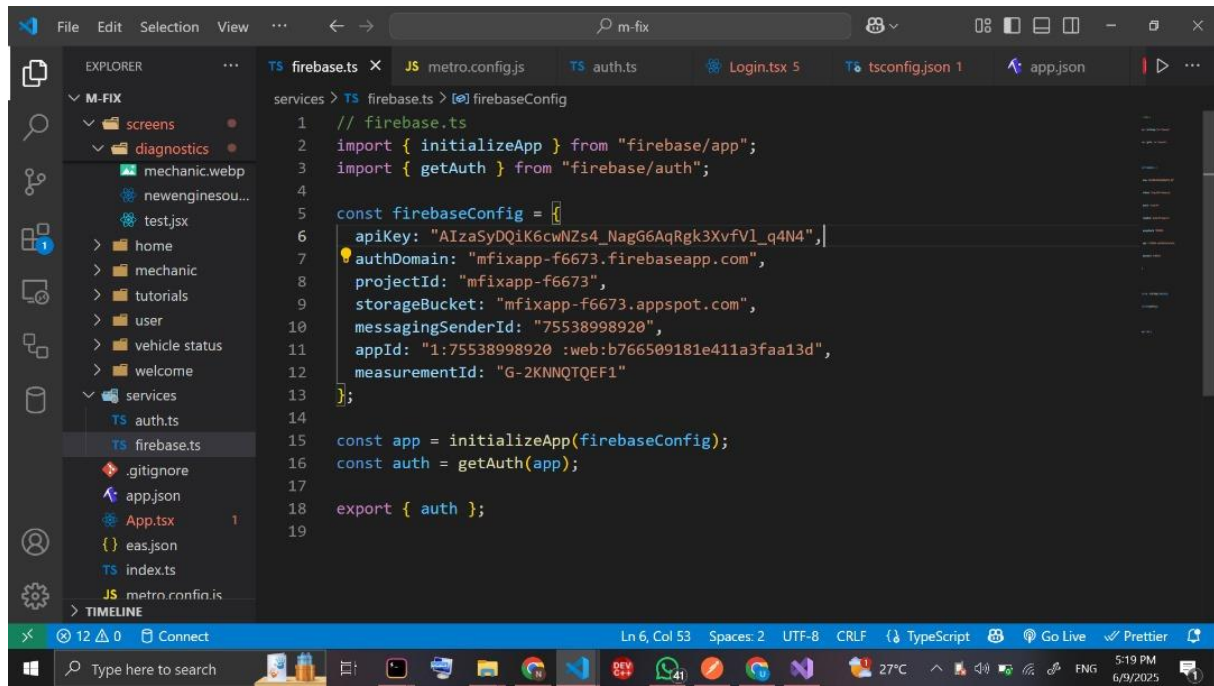
Each feature in MFix was represented as a RESTful route:

Route	Method	Purpose
/users	POST	Register new user
/users/:id	GET	Fetch user details by ID
/vehicles	POST	Add a vehicle
/diagnostics/image	POST	Analyze dashboard image
/diagnostics/sound	POST	Analyze engine sound (simulated)
/results	GET	Fetch all diagnosis results
/requests	POST	Create mechanic request
/notifications	GET	Fetch user notifications

6.3 Sample code Snippets

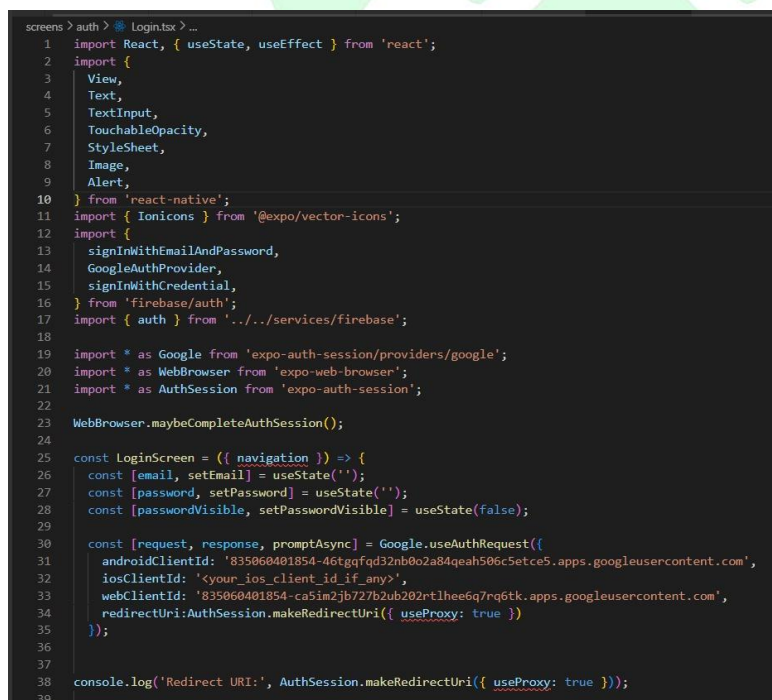
Below are real examples from the MFix backend:

i- Firebase configuration for authentication



```
services > TS firebase.ts > firebaseConfig
1 // firebase.ts
2 import { initializeApp } from "firebase/app";
3 import { getAuth } from "firebase/auth";
4
5 const firebaseConfig = {
6   apiKey: "AIzaSyDQiK6cwNZs4_NagG6AqRgk3XvfVl_q4N4",
7   authDomain: "mfixapp-f6673.firebaseio.com",
8   projectId: "mfixapp-f6673",
9   storageBucket: "mfixapp-f6673.appspot.com",
10  messagingSenderId: "75538998920",
11  appId: "1:75538998920:web:b766509181e411a3faa13d",
12  measurementId: "G-2KNNQTQEF1"
13 };
14
15 const app = initializeApp(firebaseConfig);
16 const auth = getAuth(app);
17
18 export { auth };
19
```

ii- applying the authentication in the login screen



```
screens > auth > Login.tsx > ...
1 import React, { useState, useEffect } from 'react';
2 import {
3   View,
4   Text,
5   TextInput,
6   TouchableOpacity,
7   StyleSheet,
8   Image,
9   Alert,
10 } from 'react-native';
11 import { Ionicons } from '@expo/vector-icons';
12 import {
13   signInWithEmailAndPassword,
14   GoogleAuthProvider,
15   signInWithCredential,
16 } from 'firebase/auth';
17 import { auth } from '../services/firebase';
18
19 import * as Google from 'expo-auth-session/providers/google';
20 import * as WebBrowser from 'expo-web-browser';
21 import * as AuthSession from 'expo-auth-session';
22
23 WebBrowser.maybeCompleteAuthSession();
24
25 const LoginScreen = ({ navigation }) => {
26   const [email, setEmail] = useState('');
27   const [password, setPassword] = useState('');
28   const [passwordVisible, setPasswordVisible] = useState(false);
29
30   const [request, response, promptAsync] = Google.useAuthRequest({
31     androidClientId: '835060401854-46tggfqd32nb0o2a84qeah506c5etce5.apps.googleusercontent.com',
32     iosClientId: '<your_ios_client_id_if_any>',
33     webClientId: '835060401854-ca5in2jb727b2ub202rtlhee6qrq6tk.apps.googleusercontent.com',
34     redirectUri: AuthSession.makeRedirectUri({ useProxy: true })
35   });
36
37   console.log('Redirect URI:', AuthSession.makeRedirectUri({ useProxy: true }));
38
39
```

iii- Using the Youtube API

```

1  const express = require("express");
2  const router = express.Router();
3  const { db } = require("../firebase");
4  const axios = require("axios");
5  require("dotenv").config(); // Load .env variables
6  // Fetch YouTube Video Using Fault Name
7  async function fetchYouTubeVideo(faultName) {
8    const apiKey = process.env.YOUTUBE_API_KEY;
9    const query = encodeURIComponent(`${faultName} car repair`);
10   const url = `https://www.googleapis.com/youtube/v3/search?part=snippet&type=video&q=${query}&key=${apiKey}&maxResults=1`;
11
12   try {
13     const response = await axios.get(url);
14     const videoId = response.data.items[0]?.id?.videoId;
15     return videoId ? `https://youtube.com/watch?v=${videoId}` : null;
16   } catch (error) {
17     console.error("YouTube API error:", error.message);
18     return null;
19   }
20 }

```

iv- Image diagnosis

```

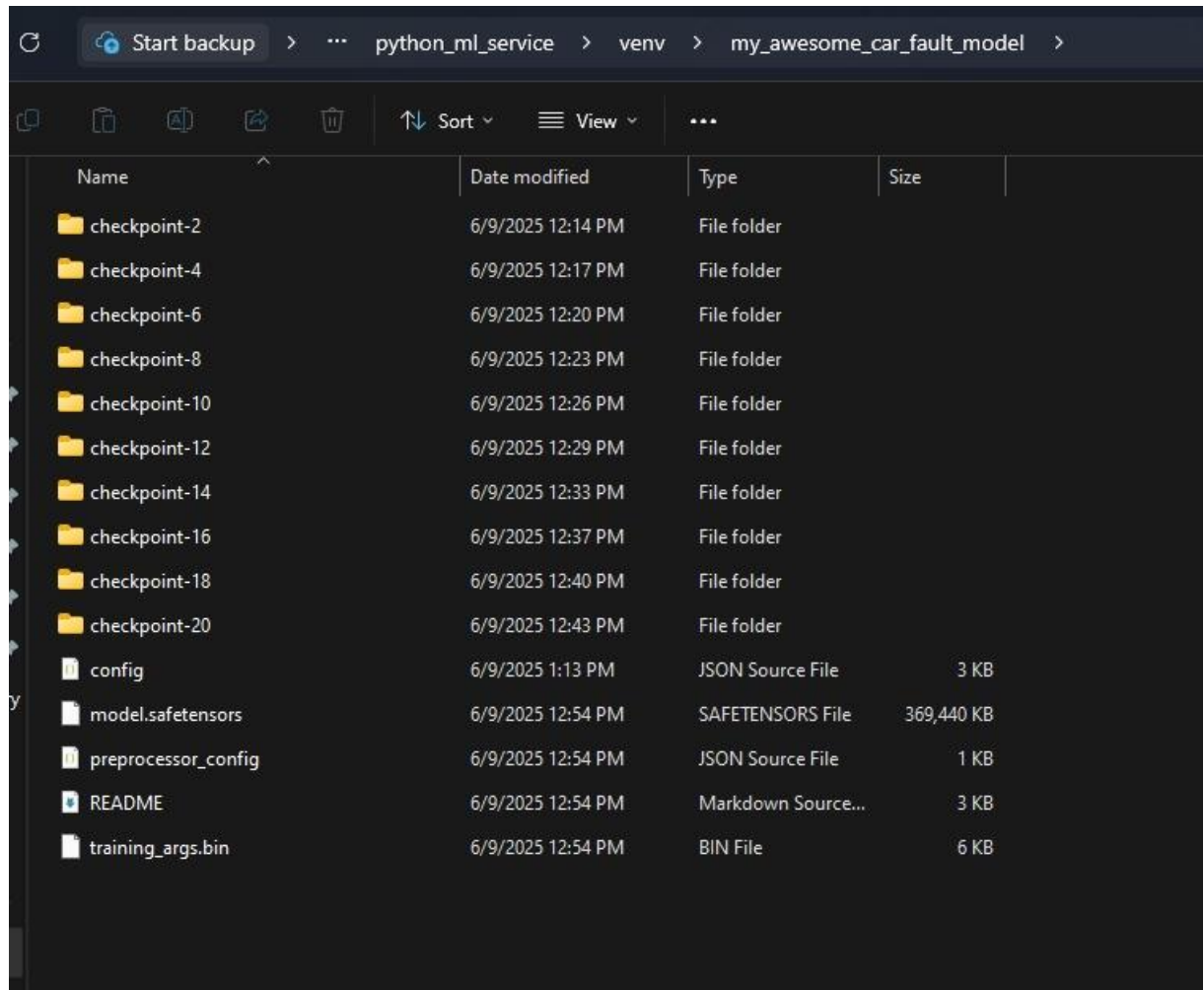
21 // POST /diagnose/image (uses Gemini API)
22 router.post("/image", async (req, res) => {
23   const { userId, vehicleId, inputData } = req.body;
24
25   if (!userId || !inputData) {
26     return res.status(400).json({ error: "Missing parameters" });
27   }
28
29   try {
30     const geminiApiKey = process.env.GEMINI_API_KEY;
31     const geminiUrl = `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${geminiApiKey}`;
32
33     const promptText = `Analyze this car dashboard warning light image. Respond ONLY with a JSON object containing the following keys:
34     "faultName": (string, concise name of the fault)
35     "explanation": (string, detailed explanation of the fault)
36     "recommendation": (string, actionable advice for repair or next steps)
37     Do not include any other text or formatting outside the JSON.`;
38
39     const payload = {
40       contents: [
41         {
42           parts: [
43             { text: promptText },
44             {
45               inlineData: {
46                 mimeType: "image/jpeg",
47                 data: inputData // base64-encoded JPEG, no prefix
48               }
49             }
50           ]
51         }
52       ],
53       // Add generation config to request JSON output
54       generationConfig: {
55         responseType: "application/json"
56       }
57     };
58
59     const geminiRes = await axios.post(geminiUrl, payload);
60
61     console.log("Gemini raw response:", JSON.stringify(geminiRes.data, null, 2)); // Debug log for raw response
62
63     const jsonString = geminiRes.data?.candidates?.[0]?.content?.parts?.[0]?.text;
64
65     if (!jsonString) {
66       throw new Error("No valid JSON response from Gemini.");
67     }
68
69     // Attempt to parse the JSON string
70     let parsedGeminiResult;
71     try {

```

v- In order to implement the sound we had to train a model capable of recognising various engine sound. This was done so as to have a more precise diagnostic.

- For the sound diagnostics component, we utilized a dataset sourced from [Kaggle.com](https://www.kaggle.com/datasets/abhishek1998/engine-sound-dataset) containing **34 labeled audio recordings of engine sounds**. These recordings represented various engine conditions, such as normal operation, knocking, and misfires.

- To develop the sound recognition model, we leveraged a pre-trained audio classification architecture available on [HuggingFace.com](https://huggingface.com). The model was fine-tuned using our dataset to adapt it specifically to vehicle engine sound patterns.
- While the dataset was relatively small, the resulting model demonstrated a **basic but functional ability to recognize and classify engine sounds**. We acknowledge the limitations in terms of dataset size and variety, but this marks a strong foundation for future improvements.



Name	Date modified	Type	Size
checkpoint-2	6/9/2025 12:14 PM	File folder	
checkpoint-4	6/9/2025 12:17 PM	File folder	
checkpoint-6	6/9/2025 12:20 PM	File folder	
checkpoint-8	6/9/2025 12:23 PM	File folder	
checkpoint-10	6/9/2025 12:26 PM	File folder	
checkpoint-12	6/9/2025 12:29 PM	File folder	
checkpoint-14	6/9/2025 12:33 PM	File folder	
checkpoint-16	6/9/2025 12:37 PM	File folder	
checkpoint-18	6/9/2025 12:40 PM	File folder	
checkpoint-20	6/9/2025 12:43 PM	File folder	
config	6/9/2025 1:13 PM	JSON Source File	3 KB
model.safetensors	6/9/2025 12:54 PM	SAFETENSORS File	369,440 KB
preprocessor_config	6/9/2025 12:54 PM	JSON Source File	1 KB
README	6/9/2025 12:54 PM	Markdown Source...	3 KB
training_args.bin	6/9/2025 12:54 PM	BIN File	6 KB

Trained model

7- CONNECTING DATABASE TO BACKEND

7.1 Database Configuration

Before the backend can interact with Firestore, Firebase must be properly configured using service credentials. In MFix, we created a Firebase project and downloaded the `serviceAccountKey.json` file containing private keys and identifiers. This file was securely loaded using Node's `fs` module.

7.2 Connection Logic in Backend

The backend uses the Firebase Admin SDK to establish a persistent connection to Firestore. This connection is initiated once, globally, in `index.js` and shared across routes using the exported `db` reference:

```
// Shared Firestore instance
const db = admin.firestore();
module.exports = db;
```

Each backend route (e.g., `users`, `diagnostics`) imports the database instance and performs operations using Firestore's document and collection APIs.

Example in `diagnostics` file:

```
const db = require("../config/firebase");

router.post('/image', async (req, res) => {
  const result = await db.collection('results').add({...});
  res.status(200).json({ resultId: result.id });
});
```

7.3 Error Handling and Logging

To ensure stability and ease of debugging, all Firestore operations are wrapped in `try-catch` blocks. Errors are logged to the console and optionally stored or sent to a monitoring service in future versions.

Example:

```
try {
  const data = await db.collection('vehicles').doc(vehicleId).get();
  res.json(data.data());
}
```

```

} catch (error) {
  console.error("Error fetching vehicle:", error.message);
  res.status(500).json({ error: "Internal server error" });
}

```

We used status codes like 400, 404, and 500 to return meaningful messages to the frontend for better user feedback and UI handling.

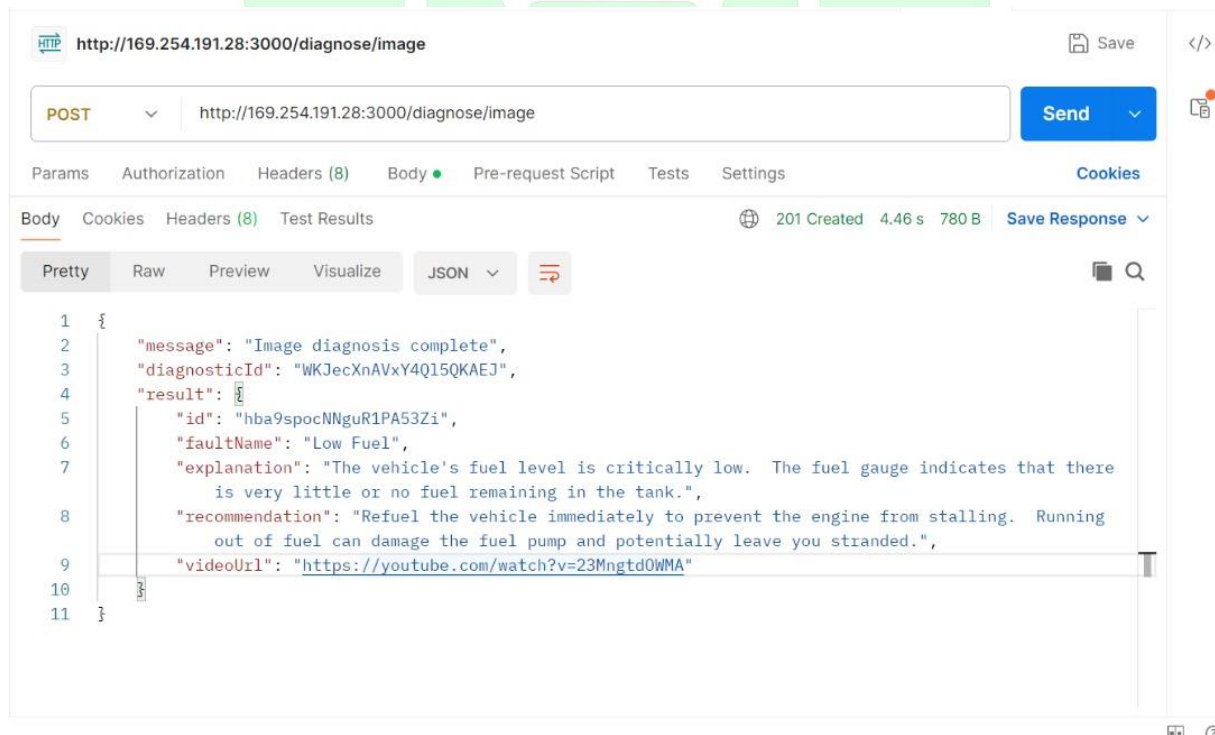
8. TESTING AND VALIDATION

Using **Postman**, we tested each backend route with valid and invalid data to simulate real-world usage. Sample tested routes include:

- POST /users – User creation
- POST /vehicles – Add vehicle
- POST /diagnostics/image – Send dashboard image for AI diagnosis
- GET /results – Fetch diagnosis result

Each test verified response status, returned data structure, and database changes.

Sample of test



Dashboard scan result

9- CHALLENGE FACED AND SOLUTIONS

1. Finding the right API to use

Challenge:

- Difficulty in identifying and integrating APIs that support sound analysis, image recognition, and vehicle diagnostics.

Solution:

- Chose APIs that are lightweight, mobile-friendly, and support REST integration with React Native. Such as Gemini API

2. Finding the Dataset for Sound Diagnostics

Challenge:

- Difficulty in locating high-quality, labeled audio datasets of various car faults (e.g., engine knocking, brake squealing, exhaust issues).

Solution:

- Conducted extensive online searches on academic repositories, Kaggle, GitHub, and automotive research datasets.
- Selected an open-source or public dataset that included labeled .wav/.mp3 files of car faults categorized by type.
- Applied preprocessing such as feature extraction (MFCC, spectrograms) for machine learning compatibility.

3. Parsing Image Data to the Model for Diagnostics

Challenge:

- Difficulty in sending image data from the React Native frontend to the backend model for inference.
- Direct file transfer created complexity in mobile environments.

Solution:

- Used **Base64 image encoding** to simplify image transfer over HTTP requests.
- Utilized <https://www.base64-image.de/> for manual testing and confirmation.
- On the frontend:
 - Captured the image using the device camera or gallery.
 - Converted the image to Base64 string.
 - Sent the encoded string to the backend API for processing.
- On the backend:

- Decoded the Base64 string back into an image.
- Passed it to the ML model for diagnostics.

10- FURTHER ENHANCEMENT AND PENDING TASKS

1. Google Sign-In Integration

- Implement a secure and user-friendly Google Sign-In flow on the frontend using **expo-auth-session** in React Native.
- Ensure seamless OAuth2.0 integration, token handling, and session persistence.

2. Backend Route Protection

- Secure backend API routes by **verifying the Google ID token** received from the frontend.
- Use libraries like `google-auth-library` (Node.js) or equivalent to validate and authenticate user identity before granting access to protected resources.

3. Push Notifications for Diagnosis Results

- Implement **real-time push notifications** to alert users when diagnostics (sound/image analysis) are complete.
- Use **Expo Notifications** or Firebase Cloud Messaging (FCM) for cross-platform support and background handling.

4. Diagnostics History Screen

- Develop a dedicated **"History" screen** in the frontend to display a list of previous sound and image diagnostics performed by the user.
- Each entry should include:
 - Date/time of diagnosis
 - Diagnostic result (with label)
 - Option to view/download image/audio used
- Optionally implement filters (e.g., date range, fault type) and search functionality.

11- CONCLUSION

The development and implementation of the backend and database architecture for the MFix vehicle diagnostic system mark a critical step toward delivering a reliable, intelligent, and user-friendly mobile application. By leveraging modern, cloud-based technologies like Firebase and Node.js, we ensured scalability, real-time performance, and secure data management without the overhead of server maintenance. The carefully structured database schema, supported by a clear conceptual design and ER modeling, lays a strong foundation for data integrity and future expansion. With RESTful APIs, robust CRUD operations, and thorough testing, the backend

not only meets the current functional requirements but is also fully prepared to integrate advanced features such as AI diagnostics, notifications, and multimedia storage. Ultimately, this backend serves as the backbone of the MFix platform, enabling seamless interaction between users, mechanics, and diagnostic data in a smart, responsive, and maintainable way.

REFERENCES

- Kaggle Datasets: <https://www.kaggle.com/>
- HuggingFace Models: <https://www.huggingface.com/>
- Firebase Firestore: <https://firebase.google.com/docs/firestore>
- Expo Auth Session: <https://docs.expo.dev/versions/latest/sdk/auth-session/>

